

Cadenas

Programación I

UNRN

Universidad Nacional
de Río Negro

08

Setiembr

e

Programación I

Clase 09: Memoria dinámica.

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Contenidos de la clase

- Repaso de punteros y parámetros por referencia.
- Estructura de la memoria ampliada: heap.
- Operadores: sizeof, *, &.
- Reserva de memoria: malloc, calloc, realloc y free.
- Punteros void y *type cast*.
- Operaciones, aritmética de punteros.
- Funciones con arreglos y memoria dinámica

Punteros a variables en memoria

- *Un puntero es una variable que contiene una dirección de memoria.*
 - Definimos un nuevo tipo de variable: un puntero '*'.

Dirección	0	1	2	3	4	5	6	7	8	9	10
Contenido	111	4	45	4	36	247	77	0	161	228	93
		pa			a			pb		b	

```
char a = 36;           //a está inicializada
char *pa = &a;         //un puntero de nombre 'pa'
char *pb = NULL;
char c = *pb;          //Segmentation Fault (SIGSEGV).
```

Punteros como argumentos: La función *swap* (intercambiar)

```
void swap(char a, char b)
{
    char temp;
    temp=a;
    a=b;
    b=temp;
}

int main()
{
    char a = 'x';
    char b = 'y';

    printf("a: %c\n",a);
    printf("b: %c\n",b);

    //intercambio:
    printf("intercambio:\n",a);
    swap(a,b);

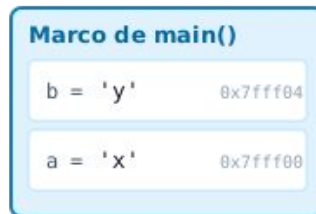
    printf("a: %c\n",a);
    printf("b: %c\n",b);

    return 0;
}
```

Direcciones ALTAS (▲)

1. Estado en main()

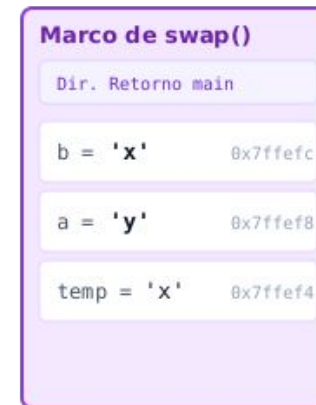
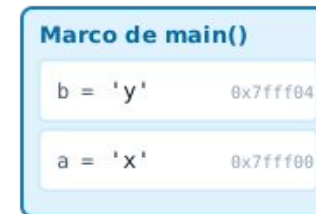
Antes de llamar a swap()



Valores originales intactos

2. Ejecutando swap()

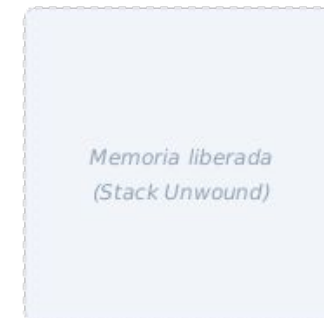
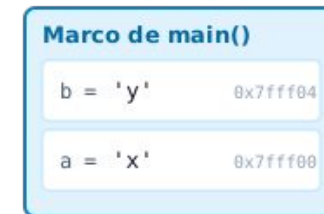
Paso por Valor (Copia en Stack)



¡Solo se cambiaron las COPIAS!

3. Retorno a main()

Frame de swap() destruido



Resultado final: SIN CAMBIOS

Direcciones BAJAS (▼)

Punteros como argumentos: La función *swap* (intercambiar)

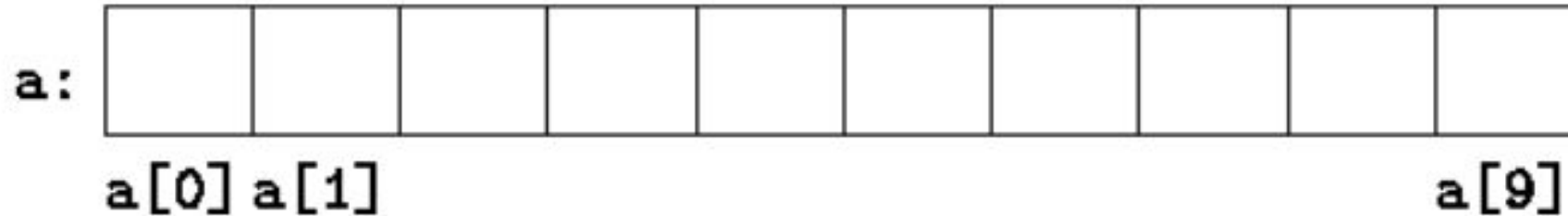
Ahora no recibe copias sino punteros,

```
void swap(char* a, char* b)
{
    char temp;
    temp=*a;
    *a=*b;
    *b=temp;
}
```



Arreglos y punteros

- Repasemos los arreglos vistos la clase pasada
 - Declaración de arreglos:
 - `int a[10];` `//arreglo de 10 enteros`



Arreglos como parámetros

- ¿Cómo definimos `sumar_elementos`?

```
int sumar_elementos(int a[], size_t nelem)
{
    //código de la función
    int suma=0;
    for(size_t i=0;i< nelem;i++)
        suma+=a[i];
    return suma;
}
```

Arreglos como parámetros

- ¿Qué pasa si modifico accidentalmente el contenido del arreglo?

```
int sumar_elementos(const int a[], size_t nelem)
{
    //código de la función
    int suma=0;
    for(size_t i=0;i< nelem;i++)
    {
        a[i]+=suma; //Ahora da error de compilación!!
    }

    return suma;
}
```

Arreglos como parámetros

- ¿Cómo definimos `sumar_elementos` usando punteros?

```
//necesitando saber la cantidad de elementos.  
int sumar_elementos(int * pa, size_t nelem)  
{  
    //código de la función  
    int suma=0;  
    for(size_t i=0;i< nelem;i++)  
        suma+=* (pa+i) ;  
  
    return suma;  
}
```

Ver [filmina](#) de aritmética de punteros.

Arreglos como parámetros

- ¿Cómo definimos `sumar_elementos` usando punteros?

```
//necesitando saber la cantidad de elementos.  
int sumar_elementos(const int * pa, size_t nelem)  
{  
    //código de la función  
    int suma=0;  
    for(size_t i=0;i< nelem;i++)  
        suma+=* (pa+i);  
  
    return suma;  
}
```

Acá también vale el const.

Arreglos como parámetros

- ¿Cómo definimos `sumar_elementos` usando punteros?

```
//necesitando saber la cantidad de elementos.  
int sumar_elementos(int * pa, size_t nelem)  
{  
    //código de la función  
    int suma=0;  
    for(size_t i=0;i< nelem;i++)  
        suma+=* (pa+i) ;  
  
    return suma;  
}
```

Ver [filmina](#) de aritmética de punteros.

Ejercicio 0 (10 minutos)

- Implementar una función que pida el ingreso de n valores flotantes por consola.

```
#include <stdio.h>
#include <math.h>

#define TAM    10

//pre: pa!=NULL && nA>0
//post: pa, vector con los valores ingresados. rv= cantidad de elementos ingresados &&
rv<=nA.
size_t ingresar_vector_f(float *pa, size_t n);

float cantidades[TAM];
int main()
{
    ingresar_vector_f();
    return 0;
}
```

Ejercicio 1 (10 minutos)

- Implementar una función que pida imprima n valores flotantes por consola.

```
#include <stdio.h>
#include <math.h>

#define TAM    10

//pre: pa!=NULL && n>0
//post: na.
void imprimir_vector_f(float *pa, size_t n);

//pre: pa!=NULL && n>0
//post: pa, vector con los valores ingresados. rv= cantidad de elementos ingresados && rv<=nA.
size_t ingresar_vector_f(float *pa, size_t n);

float cantidades[TAM];
int main()
{
    ingresar_vector_f();
    imprimir_vector_f();
    return 0;
}
```

Memoria dinámica: motivación.

- Hasta ahora todas las variables que declaramos se definen en **tiempo de compilación**.
 - ¿Qué significa **tiempo de compilación**?
- Pero hay casos en los que no podemos saber cuanta memoria necesitamos, antes de ejecutar el programa.
 - ¿Ejemplos?. Propongan ejemplos y los discutimos.
 - ¿Por qué tener una capacidad máxima para ingresar?.
 - ¿Por qué reservar memoria que no sé si voy a usar?.
- **Necesitamos una forma de adquirir memoria de forma dinámica (en tiempo de ejecución).**

Memoria dinámica

1. `malloc()` — `<stdlib.h>`

La función `malloc` (*memory allocation*) reserva un bloque continuo de memoria en el Heap.

```
void* malloc(size_t size);
```

Parámetro: `size` especifica la cantidad total de bytes a reservar.

Retorno: Un puntero genérico `void*` que apunta al inicio del bloque, o `NULL` si el sistema no pudo asignar la memoria.

Estado inicial: La memoria NO se inicializa; contiene `basura` (valores arbitrarios dejados por procesos anteriores).

```
#include <stdlib.h>

int main(void) {
    int n = 5;

    // Reserva de espacio para 5 enteros de
    // forma portable
    int *vec = malloc(n * sizeof(int));

    // Verificación obligatoria
    if (vec == NULL) {
        fprintf(stderr, "Error: No se pudo
asignar memoria.\n");
        return EXIT_FAILURE;
    }

    // Uso de la memoria...
    vec[0] = 42;

    free(vec);
    return EXIT_SUCCESS;
}
```

Memoria dinámica

2. `calloc()` — `<stdlib.h>`

La función `calloc` (*memory allocation*) reserva un bloque continuo de memoria en el Heap.

```
void* calloc(size_t num, size_t size);
```

Parámetros:

num: Número de elementos.

size: Tamaño en bytes de cada elemento.

Retorno: Puntero `void*` o `NULL` en caso de falla.

Leve costo extra por inicialización. Asegura que la memoria reservada está en 0.

```
#include <stdlib.h>

int main(void) {

    int n = 4;

    // Reserva 4 enteros garantizando que
    // inicien en 0

    int *arr = calloc(n, sizeof(int));

    if (arr == NULL) return 1;

    for (int i = 0; i < n; i++) {

        printf("arr[%d] = %d\n", i, arr[i]); //
        // Imprime 0 para todos

    }

    free(arr);

    return 0;

}
```

Memoria dinámica

3. `realloc()` — `<stdlib.h>`

Permite modificar el tamaño de un bloque previamente asignado en el Heap sin perder la información guardada

```
void* realloc(void* ptr, size_t size);
```

Parámetros:

ptr: puntero a redimensionar

size: nuevo tamaño en bytes de cada elemento.

Retorno: Puntero void* o NULL en caso de falla. Si falla no devuelve el bloque original

In-situ: Si hay espacio contiguo disponible, expande el bloque actual manteniendo la misma dirección.

Relocalización: Reserva un nuevo bloque en otra zona de memoria, copia los datos previos, libera automáticamente el bloque viejo y retorna la nueva dirección.

```
int *ptr = malloc(5 * sizeof(int));

// FORMA CORRECTA: Usar un puntero auxiliar
int *tmp = realloc(ptr, 10 * sizeof(int));

if (tmp != NULL) {
    ptr = tmp;
}

// Se actualiza la referencia de forma segura
} else {

// Si falla, 'ptr' todavía es válido y debe ser
liberado

    free(ptr);

    ptr = NULL;

    fprintf(stderr, "Error al redimensionar
memoria.\n");

}
```

Cálculo del tamaño del bloque

- malloc, calloc y realloc reciben como parámetro una cantidad de **BYTES**.
- Tenemos que calcular la cantidad de memoria necesaria conociendo la estructura de lo que vamos guardar:
 - Quiero n lugares o celdas de memoria en el vector.
 - Qiero guardar cosas de un tipo T (cada elemento “pesa” `sizeof(T)` bytes).
 - Tengo que pedir $n * \text{sizeof}(T)$ bytes.
- Ejemplos (idem calloc y realloc):
 - `char * cptr = malloc(20*sizeof(char));`
 - `int * iptr = malloc(20*sizeof(int));`
 - `float * fptr = malloc(20*sizeof(float));`

Ejercicio 2 (10 minutos)

- Implementar una función que reserve tamaño para el vector, que pida el ingreso de los valores y que luego lo imprima.

```
#include <stdio.h>

void imprimir_vector_f(float *pa, size_t n);
size_t ingresar_vector_f(float *pa, size_t n);

//float cantidades[TAM];
int main()
{
    size_t tam=0;
    float* cantidades = NULL;

    printf("ingrese la cantidad de valores:\n");
    scanf("%d",&tam);

    cantidades = ...reservar memoria...;

    ingresar_vector_f();
    imprimir_vector_f();
    return 0;
}
```

Conversión de tipos

- `malloc`, `calloc` y `realloc` tienen como tipo de retorno `void*`.
- C es un lenguaje fuertemente tipado. ¿Qué va a ocurrir si quiero un vector de enteros?

```
void* malloc(size_t size);
```

```
int *p = malloc(sizeof(int));
```

- Para forzar la asignación se necesita hacer el cast explícito.
- Sin embargo con `malloc`, la asignación se puede hacer sin hacer el cast explícito

```
int *p = (int*)malloc(sizeof(int));
```

Manejo de errores.

- Intentar acceder o desreferenciar un puntero NULL provoca un error crítico del sistema operativo: Segmentation Fault (SIGSEGV).
- Conviene armar un wrapper (envoltorio) que verifique la condición:

```
#include <stdio.h>
#include <stdlib.h>

void* asignar_seguro(size_t tam) {
    void *ptr = malloc(tam);
    if (ptr == NULL) {
        perror("[ERROR CRÍTICO] Fallo en la asignación de memoria");
        exit(EXIT_FAILURE);
    }
    return ptr;
}
```

Liberar la memoria

- La memoria estática se reserva al comienzo. Se determina en tiempo de compilación y no crecen.
 - variables static, variables globales.
- El uso de la pila es dinámico, crece con el uso y decrece. La memoria reservada en la pila se libera cuando la función termina su activación.
- La memoria del heap se comporta distinto: una vez tomada no se libera y vive mientras el proceso esté vivo. Esto requiere mucho cuidado en:
 - Devolver la memoria que no uso.
 - No perder las referencias.

Memoria dinámica

1. `free()` — `<stdlib.h>`

La función `free` libera un bloque de memoria reservado con `malloc`.

```
void free(void* ptr);
```

Parámetro: `ptr` puntero al bloque de memoria.

Retorno: NA.

Estado inicial: Libera la memoria pero no modifica el valor de `ptr`. Conviene asignar `NULL` inmediatamente después de liberar un bloque.

```
int *p = malloc(sizeof(int));  
*p = 42;
```

```
free(p); // La memoria es devuelta al  
Heap.
```

```
// 'p' ahora es un DANGLING POINTER  
(Apunta a memoria inválida/liberada).  
// *p = 100; // ;Comportamiento  
Indefinido (Undefined Behavior)!
```

```
p = NULL; // BUENA PRÁCTICA:  
Neutralizar el puntero
```

```
free(p); // free(NULL) es inofensivo y  
seguro según la especificación C
```

Aritmética de punteros.

- La aritmética de punteros NO trabaja en bytes individuales, sino en unidades del tamaño del tipo de dato al que el puntero apunta.

Si p es un puntero de tipo T^* , al sumarle un entero n :

$$\text{Dirección Resultante} = \text{Dirección Inicial} + (n * \text{sizeof}(T))$$

- Ejemplo conceptual en memoria de 64 bits:
 - Supongamos $\text{sizeof}(\text{int}) = 4$ bytes.
 - Si $\text{int}^*p = 0x1000$;
 - $p + 1$ es $0x1004$ (Avanza 4 bytes)
 - $p + 2$ es $0x1008$ (Avanza 8 bytes)

Aritmética de punteros.

Tipo	Declaración	Dirección Base (p)	Expresión $p + 1$	Dirección Final	<u>Bytes Avanzados</u>
<u>char</u>	<u>char</u> * <u>pc</u>	<u>0x2000</u>	<u>pc</u> + 1	<u>0x2001</u>	1 byte
<u>short</u>	<u>short</u> * <u>ps</u>	<u>0x2000</u>	<u>ps</u> + 1	<u>0x2002</u>	2 bytes
<u>int</u>	<u>int</u> * <u>pi</u>	<u>0x2000</u>	<u>pi</u> + 1	<u>0x2004</u>	4 bytes
<u>double</u>	<u>double</u> * <u>pd</u>	<u>0x2000</u>	<u>pd</u> + 1	<u>0x2008</u>	8 bytes

Aritmética de punteros.

- Operaciones permitidas:

- Sumar un entero a un puntero: $p + n$ o $p += n$
- Restar un entero a un puntero: $p - n$ o $p -= n$
- Incrementar y Decrementar: $p++$, $++p$, $p--$, $--p$
- Resta entre dos punteros del mismo tipo: $p2 - p1$
- Comparación de punteros: $p1 == p2$, $p1 < p2$, $p1 != p2$

- Operaciones PROHIBIDAS en Aritmética de Punteros (genera error de compilación o comportamiento indefinido):

- Sumar dos punteros: $p1 + p2$ (Sin sentido físico: ¿qué sería sumar dos direcciones de memoria?)
- Multiplicar o dividir punteros: $p1 * p2$, $p1 / 2$
- Aplicar operadores de bits: $p1 \& p2$, $p1 \ll 2$
- Aritmética sobre punteros no inicializados o NULL.
- Acceder a posiciones fuera de los límites del arreglo (Buffer Overflow).

Aritmética de punteros.

● Incremento y Decremento (++, --)

Avanza o retrocede al **siguiente/anterior elemento** en memoria contigua.

```
#include <stdio.h>

int main(void) {

    int numeros[] = {10, 20, 30, 40};

    int *ptr = numeros; // Apunta a numeros[0] (10)

    printf("Valor inicial: %d\n", *ptr); // 10

    ptr++; // Avanza a numeros[1]

    printf("Luego de ptr++: %d\n", *ptr); // 20

    ptr += 2; // Avanza a numeros[3]

    printf("Luego de ptr += 2: %d\n", *ptr); // 40

    return 0;

}
```

Aritmética de punteros.

- Precedencia de Operadores con Punteros
 - Es muy habitual combinar el operador de desreferenciación `*` con el incremento `++`. La precedencia requiere especial atención:

Expresión	Significado
<code>*p++</code>	Retorna <code>*p</code> y luego incrementa el puntero <code>p</code> .
<code>(*p)++</code>	Incrementa el valor apuntado por <code>p</code> .
<code>*++p</code>	Incrementa el puntero <code>p</code> primero y <u>desreferencia</u> la nueva dirección.
<code>++*p</code>	Incrementa el valor apuntado por <code>p</code> (equivalente a <code>++(*p)</code>).

Ejercicio: vector dinámico

- Escribir un programa que:
 - Comience y construya un vector vacío (null ptr) de tipo flotante.
 - Implemente un lazo con un pedido de un dato al usuario (inf para terminar).
 - El programa debe tener en cuenta si se el vector tiene lugar disponible.
 - Si tiene lugar debe ingresarlo al final.
 - Si no tiene debe, redimensionarlo y luego ingresarlo al final.
 - Al ingresar inf, se debe calcular la suma del vector y mostrarla en pantalla.

Preguntas disparadoras:

- ¿Cuánto espacio conviene pedir cada que vez que se llame a malloc/realloc?